

**SUPSI**

# Valutazione di WebGPU per applicazioni di Realtà Virtuale tramite integrazione con OpenXR/OpenVR

---

Studente/i

**Mazza Luca**

Relatore

**Peternier Achille**

---

Correlatore

**Paoliello Marco**

---

Committente

**Peternier Achille**

---

Corso di laurea

**Ingegneria informatica  
(Informatica TP)**

Modulo

**C11287**

---

Anno

**2025 / 2026**

---

Data

**31 maggio 2026**



*Vorrei qui ringraziare*



# Indice

|                                                        |            |
|--------------------------------------------------------|------------|
| <b>1 Abstract</b>                                      | <b>1</b>   |
| <b>2 Introduzione</b>                                  | <b>3</b>   |
| 2.1 Pianificazione . . . . .                           | 4          |
| 2.2 Requisiti . . . . .                                | 5          |
| <b>3 Stato dell'arte</b>                               | <b>6</b>   |
| 3.1 Standard didattico - OpenGL & OpenVR . . . . .     | 7          |
| 3.2 Frammentazione - Vulkan, DirectX & Metal . . . . . | 7          |
| 3.3 Comparativa - OpenVR & OpenXR . . . . .            | 8          |
| <b>4 Design e implementazione</b>                      | <b>9</b>   |
| <b>5 Test e validazione</b>                            | <b>10</b>  |
| 5.1 Risultati . . . . .                                | 10         |
| <b>6 Risultati</b>                                     | <b>11</b>  |
| 6.1 Manuale d'uso . . . . .                            | 11         |
| <b>7 Conclusioni</b>                                   | <b>12</b>  |
| 7.1 Sviluppi futuri . . . . .                          | 12         |
| <b>Glossario</b>                                       | <b>I</b>   |
| <b>Sitografia</b>                                      | <b>III</b> |

# Elenco delle figure



# Elenco delle tabelle

|     |                                  |   |
|-----|----------------------------------|---|
| 2.1 | Requisiti del progetto . . . . . | 5 |
|-----|----------------------------------|---|





# Capitolo 1

## Abstract

---



## Capitolo 2

# Introduzione

Negli ultimi anni, l'evoluzione delle API grafiche ha portato ad un progressivo allontanamento dagli standard tradizionali, come OpenGL<sup>1</sup>, in favore di soluzioni più moderne, che forniscono più controllo direttamente sull'hardware delle unità grafiche dei calcolatori. In questo panorama, WebGPU<sup>2</sup> si è imposta non solo come nuova generazione di API grafiche per il Web, ma anche come una solida alternativa per contesti nativi, grazie ai kit di sviluppo che la rendono compatibile con diversi linguaggi a basso livello, come C++ e Rust. Un progetto precedentemente sviluppato ha già analizzato e dimostrato l'efficacia della transizione da OpenGL a WebGPU per corsi introduttivi alla computer grafica.

Lo scopo del presente progetto è di estendere tale studio di fattibilità in un settore con ancora maggiori esigenze, ovvero il dominio della *Realtà Virtuale* (VR). L'obiettivo principale è la valutazione dell'impiego di WebGPU in un ambiente nativo C/C++, e che questo possa rappresentare una valida alternativa ad OpenGL per lo sviluppo di applicazioni VR, investigando la complessità e l'efficacia della sua integrazione con framework standard quali OpenXR<sup>3</sup> e OpenVR<sup>4</sup>.

Come per quanto riguarda i progetti precedentemente sviluppati, per valutare correttamente questa tecnologia è fondamentale tener sempre presente il contesto accademico in cui si andrà ad operare. Questo lavoro si rivolge in primis al modulo opzionale M-I6331Z - *Realtà Virtuale* della SUPSI, che si basa sulle conoscenze acquisite nel corso obbligatorio M-I5203 - *Grafica*. Lo scopo di tale corso è di insegnare allo studente lo sviluppo di un'applicazione interattiva in C++, interfacciandosi direttamente con le API di basso livello, senza affidarsi a game engine preconfezionati.

In questo scenario, la semplicità d'utilizzo è cruciale nel determinare la fattibilità; una soluzione troppo complessa rischierebbe di spostare inavvertitamente il focus del corso. Gli studenti e il docente si ritroverebbero a dedicare troppo tempo a districarsi tra le difficoltà di implementazione dell'API grafica, sottraendolo all'apprendimento dei concetti fondamentali della *Realtà Virtuale*. Tuttavia, lo sviluppo VR impone requisiti stringenti in termini di latenza, prestazioni e gestione diretta dell'hardware, elementi dove le API moderne come WebGPU eccellono, grazie al loro paradigma basato su pipeline esplicite.

Durante il progetto verrà pertanto sviluppato un prototipo funzionante che utilizzi WebGPU come backend grafico per il rendering di una scena VR stereoscopica. Verrà creato un layer di integrazione tra WebGPU e le API VR, affrontando e risolvendo le sfide tecniche legate all'acquisizione delle pose dei dispositivi, alla gestione delle swapchains e alla rigorosa sincronizzazione dei frame richiesta dal runtime, come ad esempio SteamVR.

---

<sup>1</sup><https://opengl.org>

<sup>2</sup><https://webgpu.org>

<sup>3</sup><https://khronos.org/openxr>

<sup>4</sup><https://github.com/ValveSoftware/openvr>

L'esito di questa implementazione permetterà infine di confrontare l'approccio WebGPU con le soluzioni tradizionali, misurando in modo oggettivo il livello di effort richiesto per lo sviluppo e le performance ottenute. I risultati raccolti forniranno delle linee guida concrete per determinare l'effettiva fattibilità e le modalità ottimali per l'adozione di WebGPU in ambito didattico e applicativo nel contesto della Realtà Virtuale.

Con questo documento non si intende però produrre una guida dettagliata per quanto riguarda l'API di WebGPU, e nemmeno all'utilizzo di OpenXR o alle funzionalità VR, bensì saranno espliciti solamente i concetti essenziali alla comprensione di questo lavoro. Se si desidera apprendere i concetti riguardanti entrambe le API menzionate si può fare riferimento alla guida per i fondamentali di WebGPU[1] e a quella di OpenXR[2].

È anche da notare che questo documento non ha lo scopo di rendere noto ogni singolo dettaglio di codice sviluppato, bensì di accompagnare alla comprensione del problema, dei passi intrapresi per cercare di risolverlo e, se finalmente è fattibile sostituire l'approccio tradizionale con WebGPU.

## 2.1 Pianificazione

Di seguito sono descritte le differenti fasi atte allo sviluppo del progetto. In quanto la natura del progetto sia sperimentale, ed infine l'obiettivo sia quello di verificare la fattibilità di utilizzo, il piano di lavoro non eccede nei dettagli o nei vincoli ma risulta completo, garantendo la flessibilità necessaria e l'immediata risposta ad eventuali problemi o deviazioni dai mezzi discussi inizialmente.

1. **Analisi WebGPU:** Analizzare l'approccio necessario per lo sviluppo di un'applicazione grafica facente uso dell'API di WebGPU.
2. **Analisi OpenXR:** Analizzare l'approccio utilizzato nei progetti precedenti (specialmente per quanto riguarda *XRBridge*<sup>5</sup>) per lo sviluppo di un'applicazione grafica che faccia utilizzo di OpenXR per fornire funzionalità per la realtà virtuale.
3. **Demo WebGPU indipendente:** Derivante dall'analisi riguardante l'API di WebGPU, sviluppare una demo che possa rappresentare un punto di partenza per lo sviluppo della demo finale. Questa deve concentrarsi sulle funzionalità dell'API WebGPU. Questa demo deve rimanere semplice e ridurre la complessità di integrazione di ulteriori funzionalità, specialmente quelle riguardanti la realtà virtuale.
4. **Demo OpenXR indipendente:** Derivante dall'analisi riguardante OpenXR, sviluppare una demo che possa rappresentare un punto di partenza per lo sviluppo della demo finale. Questa deve concentrarsi sulle funzionalità di realtà virtuale. Questa demo deve essere imperativamente concentrata sui requisiti del corso di realtà virtuale e concentrarsi ad adattare unicamente ciò che è necessario per il corso. Ulteriormente, questa deve essere integrabile nell'ambiente WebGPU, deve quindi avere un approccio modulare, il quale permetterà di rimuovere le funzionalità OpenGL e sostituirle con quelle di WebGPU.
5. **Swapchain condivisa:** Sviluppare una swapchain, condivisa tra WebGPU e OpenXR, che permetta di condividere le texture da mostrare tramite gli occhi dell'headset, tra l'API WebGPU e quella di OpenXR. Ciò si riduce nella produzione di una libreria bridge, che utilizzi uno dei "minimi comun denominatori"<sup>6</sup> tra le due tecnologie, in questo caso una tra Vulkan, DirectX o Metal.

<sup>5</sup><https://github.com/JollyCookie2000/xr-bridge>

<sup>6</sup>WebGPU si interfaccia direttamente con l'API di sistema per la rappresentazione nativa di immagini grafiche.

6. **Sincronizzazione & Rendering Stereoscopico:** Integrare la sincronizzazione del loop di rendering di WebGPU con i tempi dettati da OpenXR. Deve essere inoltre modificata la pipeline di WebGPU per renderizzare la scena due volte, basandosi sulle matrici di *projection* e *view* fornite da OpenXR per occhio sinistro e destro.
7. **Demo unificata:** Produrre una demo che unisca tutte le funzionalità richieste dal corso di Realtà Virtuale, utilizzando l'architettura prodotta nelle fasi precedenti. Questa deve dimostrare in maniera semplice e efficace i risultati ottenibili dalla combinazione di WebGPU e OpenXR.
8. **Benchmarking:** Misurare il Framerate (FPS) e la latenza della demo, utilizzando appropriati strumenti di misurazione.
9. **Valutazione effort:** Confrontare la pipeline prodotta con quelle utilizzate negli anni passati del corso; valutare le principali differenze e le difficoltà di comprensione, la verbosità e l'osticita di debug.
10. **Linee guida conclusive:** Produrre una guida finale che mostri brevemente le conclusioni tratte durante lo sviluppo e che guidi l'eventuale integrazione della pipeline nel corso.

Questo rapporto è stato scritto durante tutta la durata del progetto, parallelamente a tutte le fasi descritte sopra.

## 2.2 Requisiti

Tabella 2.1: Requisiti del progetto

| ID   | Descrizione                                                              | Note    |
|------|--------------------------------------------------------------------------|---------|
| R-00 | Deve presentare un backend WebGPU, con <code>wgpu_native</code> , in C++ | < C++20 |
| R-01 | Deve presentare l'integrazione di OpenXR per gestire funzionalità VR/XR  | -       |
| R-02 | Deve acquisire i dati di tracciamento dell'HMD                           | -       |
| R-03 | Deve gestire allocazione texture e submission duale con swapchains       | -       |
| R-04 | Deve gestire la sincronizzazione con il framerate runtime VR             | -       |
| R-05 | Deve presentare una demo completa di scena 3D, che utilizzi WebGPU       | -       |
| R-06 | Deve essere compatibile con Windows e Linux                              | -       |
| R-07 | Deve includere analisi complessità, comparata alla soluzione OpenGL      | -       |
| R-08 | Deve includere un'analisi della performance (benchmark)                  | -       |
| R-09 | Deve includere un verdetto finale e guida all'utilizzo                   | -       |

## Capitolo 3

# Stato dell'arte

Negli ultimi dieci anni, l'ecosistema dello sviluppo di applicazioni grafiche interattive ha visto intercorrere una trasformazione radicale. L'industria si è pian piano allontanata dai paradigmi tradizionali, basati su macchine a stati globali, per abbracciare interfacce di programmazione dal carattere esplicito, progettate per rispecchiare fedelmente l'architettura delle GPU multi-core. Ciò ha portato le API, contrariamente al trend seguito dalla maggior parte dell'ingegneria del software che si muove verso API di alto livello e astrazioni più vicine allo sviluppatore, a scendere sempre più di livello, avvicinandosi alla programmazione diretta dell'hardware grafico.

Parallelamente, il settore della Realtà Virtuale e Aumentata, comunemente raggruppato sotto il termine *Spatial computing* o XR, è maturato, passando da prototipi frammentari guidati da singoli vendor a standard aperti e unificati. Soprattutto per questo settore, il mercato ha avuto un moderato picco durante il periodo tra il 2020 ed il 2021, il quale ha poi visto un rapido declino, dovuto a diversi fattori, ma principalmente a scelte e underperformance dei produttori; specialmente lo spostamento dei settori di ricerca e sviluppo dei maggiori esponenti, come Microsoft e Meta, nel settore dell'Intelligenza Artificiale, in maniera affrettata (e poco considerata, visti i risultati ad oggi) ha posto fine all'evoluzione dei prodotti commerciali. Anche se il rilascio di un nuovo prodotto Apple nel settore, il prezzo imposto dalla casa di Cupertino non ha aiutato il mercato a rendere la tecnologia attrattiva.

Questo capitolo analizza le tecnologie attualmente impiegate in ambito didattico e industriale, evidenziando la crescente trasformazione dell'ecosistema grafico, le limitazioni e "l'invecchiamento" degli approcci tradizionali e la transizione architetture verso nuovi standard aperti, come OpenXR e WebGPU.

### 3.1 Standard didattico - OpenGL & OpenVR

Nel contesto accademico e della formazione in computer grafica, il corso di Realtà Virtuale ha fatto affidamento sull'utilizzo di OpenGL per il rendering e OpenVR per l'interfacciamento con gli HMD.

**OpenGL**, introdotto nel 1992, è un'API implicita basata su una macchina a stati finiti. Il suo successo decennale, specialmente nell'ambito accademico, deriva dalla curva di apprendimento che risulta relativamente dolce; l'astrazione di alto livello utilizzata dagli sviluppatori risulta semplice, in quanto nasconde al programmatore la complessa gestione della memoria video, la sincronizzazione dei thread e la sottomissione dei comandi alla GPU. Tuttavia, questa astrazione ha un costo architetturale severo: i driver di OpenGL devono eseguire una massiccia mole di euristiche in background per tradurre le chiamate high level in istruzioni comprensibili per la scheda grafica, causando un notevole driver overhead e un'imprevedibilità delle prestazioni, soprattutto del frame stuttering, difetti critici nell'ambito della realtà virtuale, dove la latenza *motion-to-photon* deve rimanere strettamente inferiore ai 20 millisecondi.

**OpenVR**, sviluppato da Valve come SDK per la piattaforma SteamVR, è stata la prima API di successo commerciale a standardizzare l'accesso a device dedicati alla realtà virtuale *PC-VR*. Nata in un periodo di forte sperimentazione, questa tecnologia è intrinsecamente legata all'ecosistema di Valve, soprattutto con i visori di produzione HTC, per i quali le due aziende hanno collaborato nella produzione. Si basa su un modello nel quale l'hardware sta al centro dell'attenzione; infatti il runtime costringe l'hardware concorrente a tradurre i propri dati per renderli compatibili con il formato di un HTC Vive, per esempio. Questa soluzione costringerebbe il programmatore a rilasciare una nuova patch ogni volta che un nuovo controller, visore o elemento periferico per il sistema VR viene rilasciato. Pur offrendo un'integrazione diretta e robusta con OpenGL, al momento si tratta di una tecnologia legacy, non più allineata con la traiettoria dell'industria moderna.

### 3.2 Frammentazione - Vulkan, DirectX & Metal

Il limite fisiologico di OpenGL ha portato alla nascita di una nuova generazione di API grafiche di basso livello; il successore naturale è **Vulkan**, sviluppato dallo stesso Khronos Group, **DirectX**, sviluppato da Microsoft con l'obiettivo di spingere il Gaming su PC, e **Metal**, di produzione Apple, per tutte le piattaforme correntemente sul mercato (iOS, macOS, tvOS, ecc...). Sebbene queste API garantiscano prestazioni eccezionali eliminando l'overhead dei driver e permettendo il multithreading nativo, hanno frammentato drasticamente il mercato, rendendo lo sviluppo multiplatforma un'impresa titanica.

Vi sono due fattori critici che rendono queste API problematiche per la didattica:

1. Vulkan richiede un approccio esplicito alla gestione delle risorse. Il developer deve allocare manualmente la memoria, gestire i descriptor set, configurare le pipeline memory barrier e orchestrare le code di comandi. Realizzare il rendering di un singolo triangolo colorato utilizzando Vulkan richiede la stesura di 1000-1500 righe di codice, che a confronto delle 50-100 di OpenGL risultano eccessive per un corso di dieci settimane. Inoltre, un corso accademico focalizzato sulla realtà virtuale, dove gli argomenti principali dovrebbero essere la stereoscopia, l'interazione spaziale e la cinematica, forzare gli studenti ad apprendere quest'API significherebbe consumare l'intero semestre solo per inizializzare l'ambiente grafico.
2. Apple ha ufficialmente deprecato OpenGL sui propri sistemi operativi, per poi rimuovere completamente il supporto nativo a tutte quelle tecnologie non-proprietarie che compongono lo standard *de-facto* dell'industria. Attualmente infatti, l'unica API grafica nativa



supportata dall'hardware Apple Silicon e dai sistemi operativi moderni della casa californiana, è **Metal**. Apple ha rifiutato di adottare Vulkan, costringendo gli sviluppatori ad affidarsi a complessi layer di traduzione, come *MoltenVK*, per ottenere una parvenza di compatibilità. Questa tendenza a murare il proprio giardino distrugge il paradigma *Write once, Compile everywhere*, rendendo impossibile fornire un ambiente di sviluppo nativo e condiviso tra chi sceglie di utilizzare Windows, Linux o macOS.

### 3.3 Comparativa - OpenVR & OpenXR

## **Capitolo 4**

# **Design e implementazione**

## Capitolo 5

# Test e validazione

### 5.1 Risultati

## **Capitolo 6**

# **Risultati**

### **6.1 Manuale d'uso**

## Capitolo 7

# Conclusioni

### 7.1 Sviluppi futuri



# Glossario

**DirectX** (in origine chiamato "Game SDK") è una raccolta di API per lo sviluppo semplificato di videogiochi per sistema operativo Microsoft Windows. Il componente DirectX Graphics permette la presentazione di grafica 2D e 3D a video, direttamente interfacciandosi con la GPU. 4, 7

**Framerate (FPS)** Frequenza di cattura o riproduzione dei fotogrammi che compongono un filmato. Un filmato, o un'animazione al computer, è infatti una sequenza di immagini riprodotte ad una velocità sufficientemente alta da fornire, all'occhio umano, l'illusione del movimento. 5

**HMD** Head-Mounted Display (in italiano schermo montato sulla testa), schermo montato sulla testa dello spettatore attraverso un casco ad-hoc e può essere monoculare o binoculare. 5

**Metal** Insieme di API grafiche e di calcolo a basso livello, sviluppate da Apple per offrire accesso diretto alla GPU dei suoi dispositivi. 4, 7, 8

**OpenVR** API e Runtime che consente accesso alle risorse hardware VR di diversi produttori, senza richiedere conoscenze specifiche dell'hardware stesso. 3

**OpenXR** Standard aperto che fornisce un set di API per lo sviluppo di applicazioni XR (realtà virtuale, mista, ecc...) compatibili con un grande range di dispositivi AR e VR. 3, 6

**Vulkan** Interfaccia programmatica di applicazione (API) di basso livello, multi-piattaforma in 2D e 3D, annunciata la prima volta al GDC 2015 da Khronos Group. Inizialmente venne presentata come "OpenGL di prossima generazione". 4, 7





# Sitografia

- [1] Gregg Travers. *WebGPU Fundamentals*. 2020. URL: <https://webgpufundamentals.org> (visitato il 28/05/2026).
- [2] The Khronos Group. *OpenXR Tutorial*. 2024. URL: <https://openxr-tutorial.com> (visitato il 28/05/2026).